real-tr-p8-full

An AgentMPI run analysis

Abstract

Eight agent ranks translated a book by the stencil decomposition the translation experiment is built around: `scatterv` the sections, extract terminology locally, `allreduce` a glossary under `UNION`, translate, reconcile seams by `halo_exchange` on a 1-D Cartesian ring, and `gather`. It completed in 166.78 s with no failures, no contract violations and no retries, at an achieved parallelism of 6.19× on eight ranks. The interesting content is in the 22.6% that did not overlap: 301.1 rank-seconds of idle time, of which 163.1 (54%) is a single pre-gather barrier waiting on one straggling revision, 55.9 (19%) is the glossary `allreduce` waiting for the last terminology extraction, and only 34.7 (12%) is the halo exchange — the collective that touches every rank costs five times what the collective that touches two neighbours costs, and that ratio is the argument for the decomposition. The run also carries the campaign's clearest evidence for the lifecycle claim: 33 worker processes served 24 agent calls across eight durable rank roles, and messages queued to a rank were delivered across a change of the process holding it. Two of those 33 workers registered as ranks 8 and 14 in a world of size 8 and were accepted; the reader should take from this run both that the decomposition works and that AgentMPI's rank registration performs no membership check at all.

1 What this run is

property	value
run	`real-tr-p8-full`
experiment	`translation`
campaign label	`real-tr`
declared world size	8
ranks appearing in the log	10
executors	`broker` × 8, `worker` × 33
events	347
wall time	166.78 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	6.19×	total busy time over wall time
parallel efficiency	77.4%	achieved parallelism per rank
idle fraction	22.6%	rank-seconds paid for and unused
peak ranks busy	8	of 8 in the world
serial fraction of active time	9.5%	time with exactly one rank busy
load imbalance	1.15×	slowest rank's busy time over the mean
coordination share	16.4%	rank-seconds blocked in collectives, of those available
coordination in flight	26.9%	wall clock with any rank inside a collective
rank reattachments	31	fresh agent processes that took over a rank role
agent calls	24	invocations that reached a model
agent latency p50 / p95	45.02 / 70.45 s	per invocation
messages	56	48 eager, 8 rendezvous
tokens sent	31,603	8,273 deferred under rendezvous
tokens in / out	43,448 / 8,636	prompt and completion
cost	$0.2599	0.0301 $/kTok out

3 What the analysis notices

- [warning] `halo_exchange/?` reports 0 messages but the fabric logged 14: the collective's own accounting disagrees with the traffic it produced.
- [warning] Ranks 8, 14 appear in the log without participating; the declared world size is 8. Shared worker pools let a worker register against the wrong job, which inflates the apparent rank count.
- [ok] 31 rank reattachment(s) occurred, up to incarnation 5: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- [ok] All 9 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	134.04	80.4	3	3	10	10	12,072	0.0	finalized
1	142.47	85.5	3	3	5	5	631	0.0	finalized
2	129.16	77.5	3	3	8	8	3,873	0.0	finalized
3	116.23	69.7	3	3	5	5	576	0.0	finalized
4	141.57	84.9	3	3	11	11	9,000	0.0	finalized
5	148.05	88.9	3	3	5	5	724	0.0	finalized
6	104.34	62.6	3	3	8	8	4,075	0.0	finalized
7	117.32	70.4	3	3	4	4	652	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

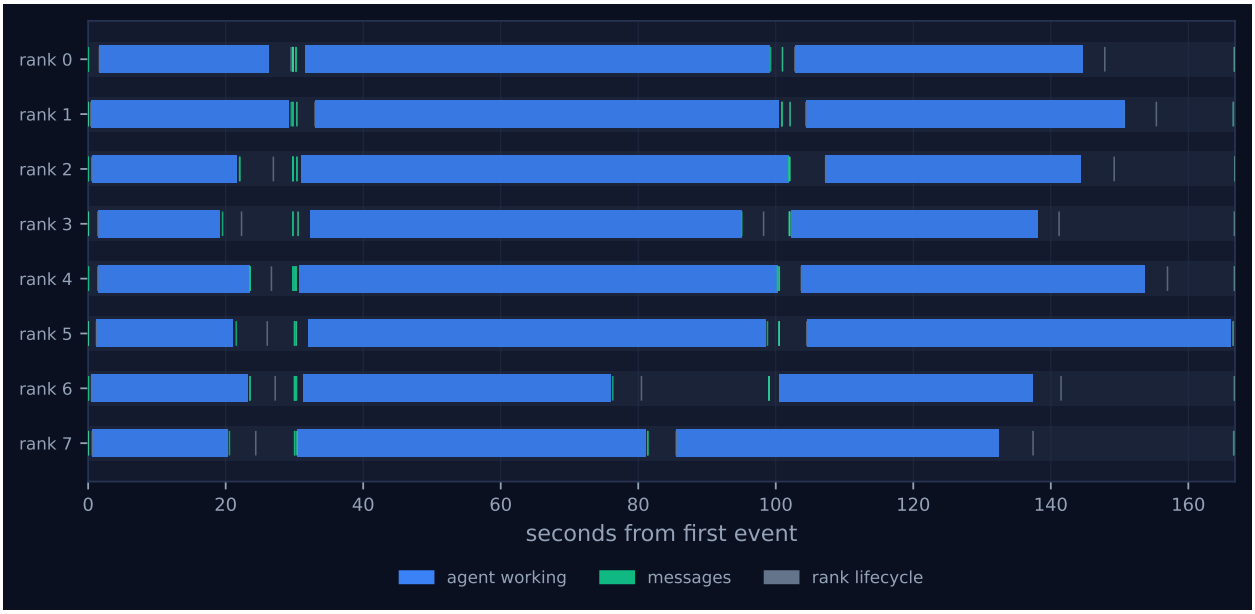

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

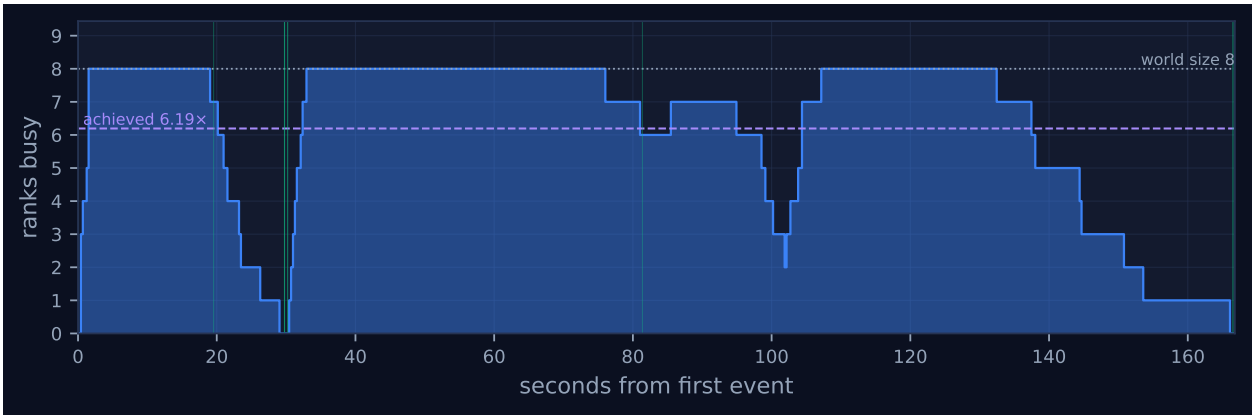

Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

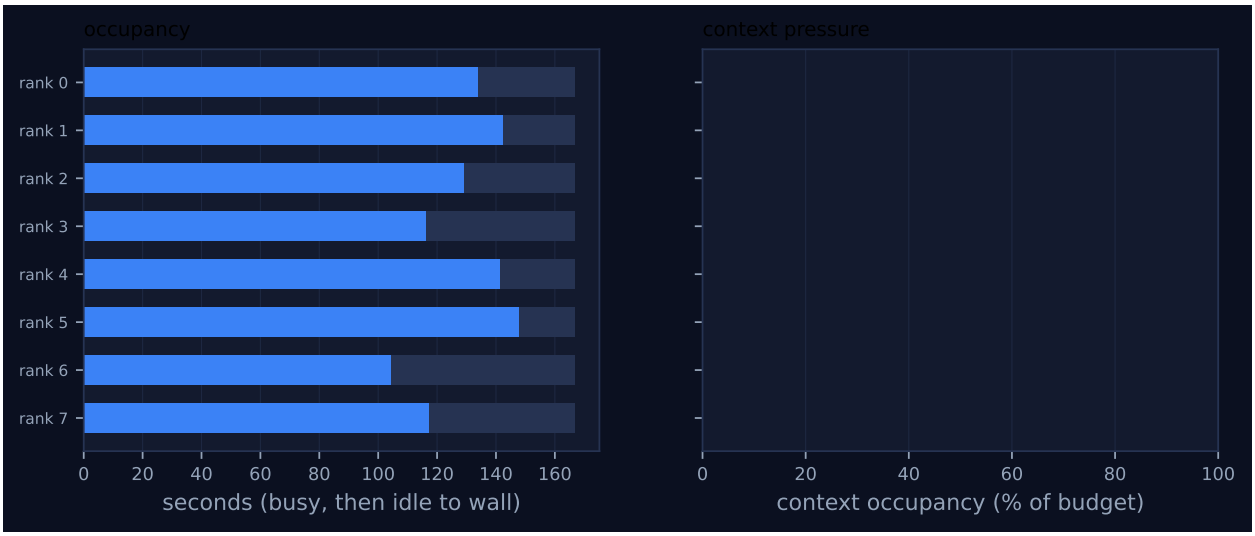

Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
scatter	binomial	decompose	8	8	3	3	7	7	11,594	0.03	0.02	✓
reduce	binomial	glossary	8	8	3	3	7	7	2,504	3.23	10.22	✓
bcast	binomial	glossary	8	8	3	3	7	7	11,123	10.24	0.26	✓
allreduce	reduce_bcast	glossary	8	8	6	6	14	14	0	10.24	0.26	✓
reduce	binomial	register	8	8	3	3	7	7	260	0.45	0.44	✓
bcast	binomial	register	8	8	3	3	7	7	1,127	0.75	0.31	✓
allreduce	reduce_bcast	register	8	8	6	6	14	14	0	0.75	0.31	✓
halo_exchange	?	seams	8	8	0	–	14	–	0	0.00	20.70	–
barrier	central	pre-gather	8	8	0	2	0	0	0	33.70	0.25	✓
gather	binomial	collect	8	8	3	3	7	7	4,456	0.09	0.26	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

The timeline in Figure 1 is three blue bars per rank and almost nothing else. Those bars are the three agent calls each rank makes — terminology extraction, translation, seam revision — and they account for 1,033.19 of the 1,334.27 rank-seconds the run had available. Everything the protocol did occupies the thin gaps between them, and the shape of those gaps is the whole story.

The first gap is a synchronisation. Terminology extraction finished at wildly different times: rank 3 at 19.54 s, rank 1 at 29.54 s, a spread of 10.0 s on calls whose median latency was 22.76 s. Every rank

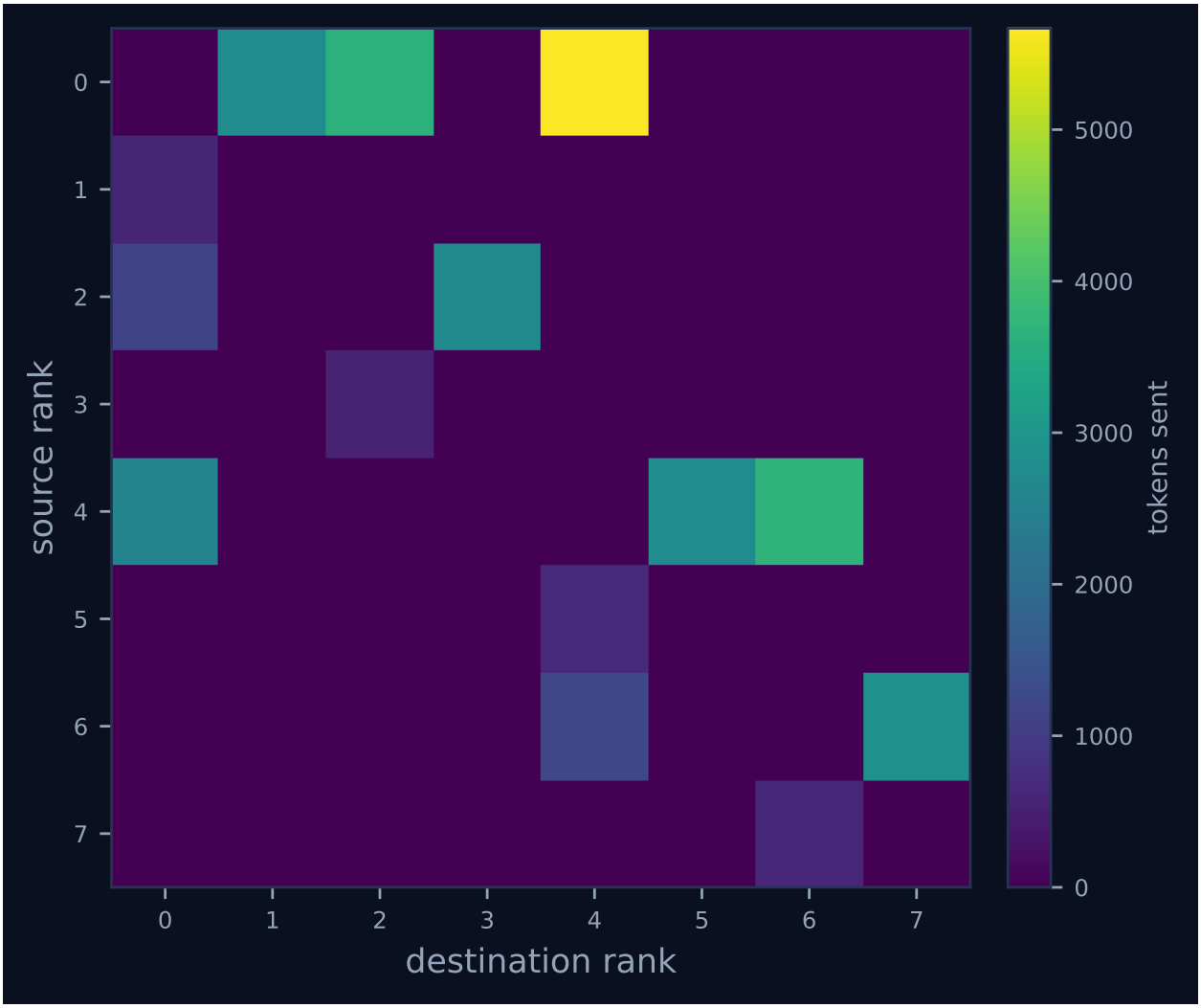

Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

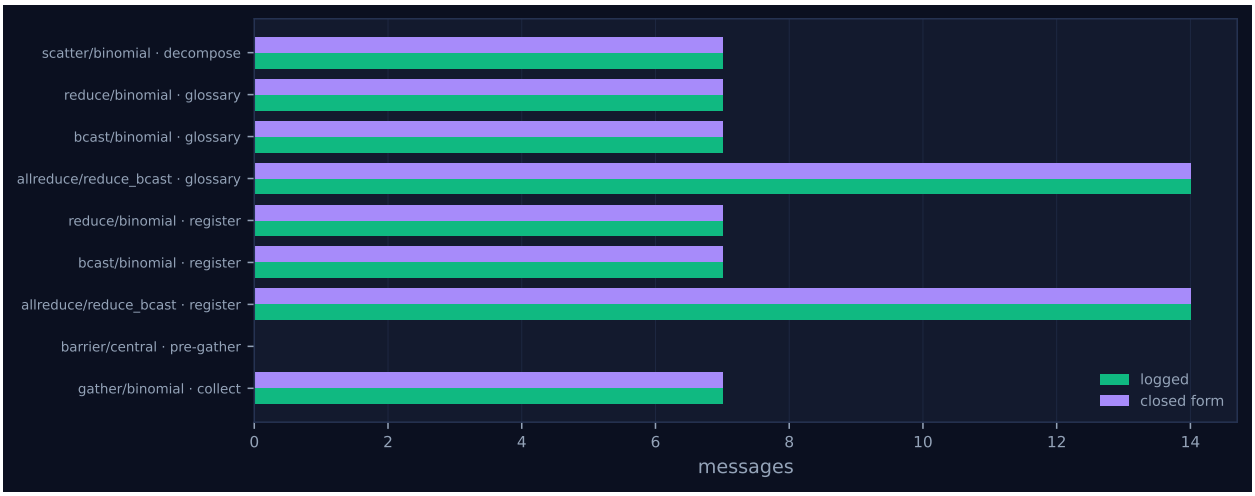

Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

on finishing enters `allreduce(local_terms, UNION)`, which under `reduce_bcast` is a binomial reduce to rank 0 followed by a binomial broadcast back. The reduce's first contribution entered the tree at $t = 19.545\,\mathrm{s}$ and its skew is $10.217\,\mathrm{s}$; the root could not fold until rank 1's message arrived, and the trace shows exactly that — rank 0 sat on partial contributions from ranks 2 and 4 for $7.72\,\mathrm{s}$ and $6.22\,\mathrm{s}$ of queue residency before rank 1's 208-token term sheet let it complete at $t = 29.76\,\mathrm{s}$. The broadcast that followed took $0.26\,\mathrm{s}$ of skew and $10.24\,\mathrm{s}$ of nominal wall, of which all but a quarter of a second is the reduce it waited on. In Figure 2 this is the first dip, and it is the deeper of the two: concurrency falls monotonically from eight busy ranks at $t = 19.2\,\mathrm{s}$ to *zero* between $t = 29.2$ and $t = 30.3\,\mathrm{s}$, and does not recover to eight until $t = 32.8\,\mathrm{s}$. For 1.1 seconds this eight-rank job had nothing running anywhere. That is the price of an exact global collective, paid in full and visible.

The second dip, from $t = 76.2$ to $t = 107.0\,\mathrm{s}$, is the halo exchange, and it is structurally different: it bottoms out at three ranks busy, not zero. Translation completions were spread over $25.9\,\mathrm{s}$ (rank 6 at $76.30\,\mathrm{s}$, rank 2 at $101.88\,\mathrm{s}$), a far worse spread than the terminology phase, so a global barrier here would have been expensive. It was not a global barrier. Each rank blocked only on its two ring neighbours, and the per-rank block times measured from the end of the translation call to the exit from `halo_exchange` are 0.00, 0.17, 0.24, 1.20, 1.70, 1.73, 6.96 and $22.72\,\mathrm{s}$ — total $34.72\,\mathrm{s}$, of which rank 6 alone contributes 22.72. Rank 6 finished first and then waited for rank 5, its left neighbour, which finished $22.5\,\mathrm{s}$ later; its right neighbour rank 7 had replied at $t = 81.38\,\mathrm{s}$ and that reply sat matched-but-unread in rank 6's mailbox for $17.64\,\mathrm{s}$ while rank 6 waited on the other side. Rank 7, at the non-periodic right boundary, had only one neighbour and cleared the exchange in $0.00\,\mathrm{s}$; it claimed its revision at $t = 85.53\,\mathrm{s}$, $16.35\,\mathrm{s}$ before the last translation in the population even finished. That head start is the measurable dividend of choosing a neighbourhood collective over a barrier, and Figure 1 shows it plainly as rank 7's third bar starting well to the left of everyone else's.

The third feature is not a dip but a drain. From $t = 132.6\,\mathrm{s}$ the concurrency curve walks down a staircase — 8, 7, 6, 5, 4, 3, 2, 1 — and holds at one busy rank from $t = 153.7\,\mathrm{s}$ to $t = 166.2\,\mathrm{s}$. This is the `pre-gather` barrier. Revision latencies ranged from $36.51\,\mathrm{s}$ (rank 3) to $66.02\,\mathrm{s}$ (rank 5), and the barrier's cost is set by the maximum, not the mean: the per-rank waits are 0.00, 12.69, 15.44, 21.70, 22.20, 28.20, 29.20 and $33.71\,\mathrm{s}$, summing to $163.14\,\mathrm{s}$. Seven ranks spent between 12.7 and 33.7 seconds doing nothing so that rank 5 could finish one revision. The barrier is followed by the

`gather`, which is over in 0.092 s — collecting the finished book costs essentially nothing next to waiting to be allowed to collect it.

The whole idle budget decomposes cleanly, and it closes:

where the idle time went	rank-s	share
`pre-gather` barrier	163.14	54.2%
glossary `allreduce`, both invocations	55.87	18.6%
broker dispatch, enqueue to claim	39.16	13.0%
`halo_exchange`	34.72	11.5%
completion polling, `gather` and finalize	8.19	2.7%
total idle, of 1,334.27 rank-s available	301.08	100.0%

These were computed from the event log: busy spans are `broker.claim` to `broker.complete`, collective block times are the interval from a rank's preceding `agent.call` to its `coll.*` event, and dispatch latency is `broker.enqueue` to `broker.claim`. The last row deserves a note, because it is measurement apparatus rather than protocol: the 24 summed `agent.call` latencies total 1,079.37 s against 1,033.19 s of claim-to-complete work, and the 46.18 s difference is accounted for by the 39.16 s of dispatch plus the 6.97 s a rank takes to notice its task finished. Every one of the 24 recorded latencies lies within 25 ms of a multiple of 0.5 s, and the observation lag never exceeds 0.500 s, so the headline "agent latency p50 46.02 s" is a polled quantity quantised at half a second and inflated by AgentMPI's own scheduling. The model was working for 95.7% of it.

Two other things are worth reading off the pictures. Figure 4 is extraordinarily sparse: of 56 ordered rank pairs only 20 carry any traffic, and they are exactly the 14 directed edges of the binomial tree over eight ranks plus the six directed edges $\{1,2\}$, $\{3,4\}$, $\{5,6\}$ that the ring adds and the tree does not already contain. The single brightest cell is $0 \to 4$, 5,669 tokens over three messages, dominated by the 3,919-token scatter hop that carries half the book. There is no all-to-all anywhere in the plane. And Figure 3 has an empty right-hand panel: context occupancy is 0.0% for every rank against a 120,000-token budget, because every receive in this harness is non-admitting and the `gather` is issued with `admit=False`. Nothing this run did touched an agent's context window.

The viewer screenshot shows one thing the figures do not, and it is the reason to look at both. The dashboard draws ten lanes. Ranks 0–7 carry their three work bars each; ranks 8 and 14 carry a single grey lifecycle tick at the far left and are otherwise empty for the full 166.8 s. Figure 1 draws eight lanes, because `analyze_run` filters to participating ranks. The plot is the more useful picture of the computation and the screenshot is the more honest picture of the job.

7 Interpretation

What is true by construction

The phase structure is the harness's, not a discovery. `translation.py` calls `scatterv`, then one `agent` per unit, then two `allreduce`s under `UNION` (one for the glossary, one for the register string), then a translation call, then `cart_create` plus `halo_exchange`, then a revision call, then `barrier` with `PROCEED` and `gather`. The trace matches instruction for instruction: 8 `coll.scatter`, 16 `coll.reduce`, 16 `coll.bcast`, 16 `coll.allreduce`, 8 `topo.cart_create`, 8 `coll.halo_exchange`, 8 `coll.barrier`, 8 `coll.gather`, and 24 `agent.call`. That the glossary is identical at every rank

is likewise by construction and not evidence: the broadcast payload digest is `78212137b9f7` on all seven messages that carry it, so all seven non-root ranks received the byte-identical glossary the root folded, which is what an exact, associative, commutative and idempotent operator over a tree guarantees. It would be a defect if it were otherwise; it is not a finding that it is not.

Nine of the ten collective invocations sent exactly the number of messages their closed-form cost expression predicts, which Figure 5 shows as eight matched pairs of bars. The `barrier` row is empty in that figure because a central barrier is fabric-mediated and sends no messages at all; its `rounds` field reads 0 against a predicted 2, which is a definitional mismatch in what "round" means for a non-message-passing barrier rather than a discrepancy in behaviour.

The two zeros that are not zeros

Both `allreduce` invocations report `messages_sent: 0` and `tokens: 0` in their own events, and the viewer's collectives panel faithfully prints *0* in the messages column of the `allreduce` row. This is correct and a reader should not be misled by it. Under `reduce_bcast` the `allreduce` does no communication of its own; it invokes a nested `reduce` and a nested `bcast`, each of which reports its own traffic. The trace confirms the delegation directly: rank 0's `coll.reduce` for label `glossary` at $t = 29.76\,\mathrm{s}$ is followed 0.01 s later by its `coll.bcast` and then by the `coll.allreduce` that wraps them, and the analysis attributes 14 nested messages to the wrapper on that basis — which is the 14 the cost model predicts for $2\lceil \log_2 8 \rceil$ rounds. The honest cost of the glossary `allreduce` is the 2,504 tokens of its reduce plus the 11,123 tokens of its broadcast, 13,627 in total, and of the register `allreduce` 260 plus 1,127. The zero means "delegated", not "free", and the generated collectives table quietly does the right thing for messages while still printing 0 for tokens.

The `halo_exchange` zero is different and is a real reporting defect, though a closed one. Its events carry only `{label, left, right}`; there is no `algorithm`, no `rounds`, no `messages_sent`. The fabric nonetheless logged 14 messages tagged `_ampi:halo:0:8:L` and `_ampi:halo:0:8:R`, which is exactly $2(p-1) = 14$ for a non-periodic 1-D ring, and the analysis flags the disagreement. Reading the source resolves it: `topology.halo_exchange` now emits `algorithm="sendrecv"`, `rounds`, `messages_sent` and `tokens_sent`, and it acquired them in commit `ee21d79`, "Report messages and tokens from the neighbourhood collectives", which post-dates the commit that committed this trace. So the warning is genuine, was genuine when the run was taken, and has since been fixed — but the cost model still has no closed form registered for `halo_exchange`, which is why its row shows – and why Figure 5 omits it entirely. The obvious closed form is $2(p-1)$ messages in 2 rounds, and this run's traffic agrees with it.

A rank is durable; the process holding it is not

This is the finding the run makes best. There were 41 `rank.init` events: eight with `executor: broker` at $t \approx 0$, and 33 with `executor: worker`. The 33 decompose exactly. Twenty-four of them are one worker process per agent call: rank 0's incarnations 2, 3 and 4 claimed `terms:u0`, `translate:u0` and `revise:u0` respectively, and the same pattern holds for all eight ranks. Seven are trailing attachments that found no work left — incarnation 5 on ranks 0, 1, 2, 3, 4, 6 and 7, all registered after that rank's last task completed. Rank 5 has no such trailing attachment, which is precisely why it reports 3 reattachments where every other rank reports 4: rank 5 was the barrier straggler and finished 0.63 s before the job ended, leaving no time for a spare worker to attach. Twenty-four plus seven is 31, the reported reattachment count, and the remaining two workers are

ranks 8 and 14.

What persisted across those transitions is the rank, and the trace shows it rather than asserting it. Rank 0's registration went from incarnation 2 to incarnation 3 at $t = 29.51$ s. Rank 2 had sent it a 387-token reduce contribution at $t = 22.04$ s and rank 4 an 872-token one at $t = 23.55$ s; both were matched at $t = 29.76$ s with logged queue residencies of 7.721 s and 6.217 s, spanning the incarnation boundary. The messages were addressed to *rank 0*, sat in the fabric's message table, and were still there for whoever held the rank when the receive was posted. Rank 6 shows the same across its incarnation 3 to 4 transition at $t = 80.47$ s: it sent its right boundary at $t = 76.30$ s and did not complete the exchange until $t = 99.02$ s. Eight rank identities, 33 processes, zero lost messages and zero contract violations. The SQLite `ranks` table at the end of the run records incarnation 5 for seven ranks and 4 for rank 5, with each rank's token and cost totals accumulated across all of its incarnations.

One qualification, and it matters. The claim usually made is that the rank role, its mailbox *and its context account* persist. The first two are demonstrated here; the third is not, because the context account held nothing to persist. Every `rank.finalize` reports `used: 0`, `high_water: 0`, `admissions: 0`. This run shows that a durable mailbox survives process turnover; it says nothing about whether a durable context account does.

Ranks 8 and 14: a protocol conformance defect, not just untidiness

The declared world size is 8 and the log contains ranks 8 and 14. Each emits exactly one `rank.init` — rank 8 at $t = 0.24$ s, rank 14 at $t = 1.55$ s, both `executor: worker`, both incarnation 1 — and then never sends, receives, claims, or finalizes. The framing offered is that experiments shared a persistent worker pool and a worker registered against the wrong job, and the sibling runs confirm it beyond doubt. `real-tr-p2-full`, world size 2, contains inert registrations for ranks 2, 3, 4, 5, 6, 7, 8 and 14. `real-tr-p4-full`, world size 4, contains them for ranks 4, 5, 6, 7, 8 and 14. The same pool, the same indices, three runs; every worker whose index fell outside the world it attached to registered successfully and idled.

The question the contract asks is whether AgentMPI accepting that registration is a defect. It is, and the evidence is in the source rather than only in the log. `create_job` writes `world_size` into the fabric's `meta` table and populates `comm_members` for ranks $0 \ldots p - 1$; the docstring is explicit that the world size is static in the MPI sense. `RankRuntime.register` then consults neither. It selects the `ranks` row for its own `wrank`, and on finding none performs an unconditional `INSERT`, assigns incarnation 1, and emits `rank.init`. Any process that can open the fabric can create a rank row for an arbitrary index. In MPI this is `MPI_ERR_RANK` and the call fails; here it succeeds silently.

In this run the blast radius was nil, and it is worth being precise about why: collectives resolve membership through `comm_members`, not through the `ranks` table, so ranks 8 and 14 were never addressed by any tree and never appeared in any `absent` list. Containment was real. But the consequences are not purely cosmetic even here. The final `ranks` table holds ten rows for an eight-rank world, two of them in state `running` rather than `finalized`, with leases renewed to 30 minutes past the end of a job that has already emitted `job.finish` — a completed job that a lease scanner would read as having two live ranks. And the failure mode this run happens to dodge is not exotic: registration is an upsert keyed on rank index, so a worker misdirected onto an index that is *inside* the world would not have created a new row, it would have bumped the incumbent's incarnation, taken its lease and been handed its mailbox. The durability that makes reattachment work is the same mechanism that makes a misdirected registration dangerous, and the only thing

standing between the two is a bounds check that is not there. The fix is small — compare `wrank` against the `world_size` meta key, or require membership in `comm_members` for the world context, and refuse otherwise — and I would flag it as worth making.

Transport: one message chose rendezvous, seven were told to

The headline "48 eager, 8 rendezvous, 8,273 tokens deferred" conceals a distinction the log makes clearly. `_choose_mode` selects rendezvous only when a payload exceeds the sending rank's `eager_limit`, which every `rank.init` reports as 2,048 tokens. Exactly one message in the run crossed it: the root's first scatter hop, `mid 1`, 3,919 tokens from rank 0 to rank 4 at $t = 0.01$ s. Every other eager message in the run is under the limit, the largest being 1,934 tokens. The other seven rendezvous messages are the `gather`, and they are rendezvous because the harness says `mode=Mode.RENDEZVOUS, admit=False` outright; their payloads are 313 to 1,564 tokens and under `AUTO` every one of them would have gone eager. So 3,919 of the 8,273 deferred tokens were deferred by the transport's own size rule and 4,354 by the harness's explicit instruction.

Which message crosses the limit is a property of the tree, not of the data. A binomial scatter's hop at depth k carries $p/2^{k+1}$ sections, so the root's first hop carries half the book (3,919 tokens here), the next 1,888 and 1,934, the next 930 to 1,033. Only the first is above 2,048, and only the first went rendezvous.

Under eager-only transport this run would still have completed. Each rank publishes an `unexpected_limit` of 16,384 tokens — eight times the eager limit, confirmed in the fabric's `ranks` table — and `_await_eager_credit` blocks a sender only when the destination's outstanding unmatched eager volume would exceed it. The largest single payload in the run is the 3,919-token scatter hop and the largest multi-message fan-in is the root's at gather time, 327 + 727 + 1,564 = 2,618 tokens; both are far under 16,384. No credit stall would have occurred, and indeed no `transport.credit_stall` event appears in the log. What rendezvous bought at this scale was therefore not liveness but the invariant: no rank's mailbox ever held a payload it had not asked for. The scale at which it becomes liveness is computable from the same numbers — the root's first hop is half the book regardless of p, and `_await_eager_credit` raises `AmpiContextOverflow` outright when a single payload exceeds 16,384 tokens, so an eager-only scatter fails once the book passes roughly 32,800 tokens. This book is about a quarter of that. I flag the last sentence as extrapolation from the transport code and this run's measured hop sizes, not as a measurement.

An accounting defect: `tokens_deferred` means two things

Chasing the deferral numbers turned up a genuine bug. The `job.finish` event reports `tokens_deferred: 39876`. `metrics.json`, the viewer's stats row and `cost.summarise` all report 8,273. Both are computed from the same run and they differ by a factor of 4.8.

The 8,273 is right: it is the sum of the eight rendezvous `msg.send` token counts, and it is what `cost.summarise` accumulates when it sees `mode == "rendezvous"` on a send. The 39,876 is 31,603 plus 8,273, where 31,603 is the token total of *all 56* messages. The cause is that `RankCost.tokens_deferred` is incremented in two places in `comm.py`: once on the send side when the chosen mode is `RENDEZVOUS`, and once on the receive side whenever `admit` is false. Because this harness never admits anything, all 56 receives took the second branch, and the eight rendezvous messages were counted twice — once as sent-deferred and once as received-unadmitted. The field's own docstring says "tokens that a rendezvous transfer avoided moving into context", which is the

first meaning; the receive-side increment implements the second, "tokens not admitted", and these are not the same quantity. The per-rank `rank.finalize` snapshots carry the inflated figure too: rank 0 reports 8,218 deferred tokens against 12,072 sent, none of which it sent under rendezvous except its one scatter hop.

This one is worth fixing because the deferral figure is described in the source as the headline number for the transport-mode experiment, and a headline number that reads 4.8 times high in the job totals while reading correctly in the summariser is exactly the kind of divergence that survives review. The analysis pipeline happens to use the correct path, so no figure in this document is affected — but `job.totals()` is what the experiment harness writes into `results/`.

What this point contributes to its siblings

Against the scaling series, `real-tr-p8-full` is the last configuration that works. Wall times are 576.7 s at $p = 1$, 410.1 s at $p = 2$, 308.4 s at $p = 4$, 166.8 s at $p = 8$, and 7,200.2 s at $p = 16$, where the last is the two-hour job timeout: that run made only 10 agent calls before it expired and produced a book with coverage 0.0. End-to-end speedup at $p = 8$ over the sequential baseline is therefore $576.7/166.8 = 3.46\times$, and it is important not to confuse that with the $6.19\times$ achieved parallelism in the headline table. The two differ because they measure different things and because the parallel run does more work: at $p = 1$ the harness skips the halo entirely (`cfg.halo and comm.size > 1`) and makes 16 agent calls, where $p = 8$ makes 24. Parallelising this book bought $3.46\times$ wall-clock for $1.5\times$ the agent calls and $1.61\times$ the money ($0.2599 against $0.1612). The seam reconciliation is not free, and the speedup number should always be quoted with the cost number beside it.

The scaling series also shows why the barrier dominates the idle budget and why it will get worse. Median agent latency across $p = 1, 2, 4, 8$ is 42.02, 44.52, 49.52 and 46.02 s — flat — but p95 is 55.02, 62.52, 65.02 and 70.02 s, up 27% from $p = 1$ to $p = 8$. A barrier costs the maximum of p draws, so a widening tail and a growing p compound, and the 33.7 s worst-case wait here is the mild version of that.

Against the ablations, this run is the expensive arm. Removing the glossary (`real-tr-p8-noglossary`) drops to 94.6 s, 16 calls and $0.1355; removing the halo (`real-tr-p8-nohalo`) to 125.9 s, 16 calls and $0.1628; switching the `allreduce` to recursive doubling (`real-tr-p8-rdallreduce`) to 158.1 s at essentially the same parallelism, $6.21\times$ against $6.19\times$; and replacing `UNION` with a semantic merge (`real-tr-p8-semanticgloss`) costs 1,152.9 s, a $6.9\times$ slowdown, at 15.2% efficiency. This run is what the full machinery costs, and the number it establishes is that the full machinery costs $1.76\times$ the no-glossary arm and $1.32\times$ the no-halo arm in wall time.

The recursive-doubling comparison is worth one more sentence, because it isolates the algorithm from the straggler. In this run the glossary `allreduce` under `reduce_bcast` takes $2\lceil \log_2 8 \rceil = 6$ rounds and $2(p-1) = 14$ nested messages and completes in 10.238 s; in the sibling it takes recursive doubling's $\lceil \log_2 8 \rceil = 3$ rounds and $p \log_2 p = 24$ messages and completes in 11.358 s. Halving the round count while adding 71% more messages made the collective 1.1 s *slower*. Round depth is not what this operation is paying for — the 10.217 s of arrival skew is — and any tuning that optimises the tree without addressing the spread in agent latency is optimising the wrong term.

What it does not establish — and this is the most important thing in the section — is that the machinery was worth it. The deterministic quality metrics are identical across every $p = 8$ arm: coverage 1.0, consistency 1.0, `n_divergent_entities` 0, `rendering_verified` 1.0, paragraph fidelity 1.0, in the full run, in the no-glossary run, in the no-halo run and in the semantic run alike. The consistency metric is the one the glossary machinery exists to move, and with six shared

entities across eight sections it reads 1.0 whether or not the glossary was exchanged at all. On the evidence of this campaign the eight ranks agreed on every shared rendering by themselves. This run's contribution to the ablation ladder is a clean cost measurement attached to a benefit that the metrics could not detect.

8 Threats to this reading

The metrics saturate, so the baseline may be measuring nothing. Every quality figure is at its ceiling in every arm. Six shared entities across eight 600-word sections of Sherlock Holmes is a tiny consistency surface, and `consistency` is computed only over entities that more than one unit reported. The correct reading is not "the glossary is unnecessary" but "this instrument cannot tell". Anything in this document that implies the full configuration is or is not worth its 1.76× cost is unsupported; only the cost side is measured.

The idle decomposition is a partition of intervals, not of causes. I attributed each gap to the collective the rank was inside during it, which is well defined — the boundaries are `agent.call` and `coll.*` events — but "rank 7 was idle for 33.71 s in the barrier" is measurement while "rank 7 was idle because rank 5's revision ran long" is inference. It is a well-supported inference (rank 5's revision latency of 66.02 s is the maximum of the eight, and every other rank left the barrier within 0.25 s of rank 5 entering it) but it is inference. The 39.16 s of dispatch latency in the same table is not protocol at all; it is broker scheduling, and folding it into an "idle" total alongside collective waiting flatters neither and conflates two different costs.

The concurrency curve is sampled, and so are the dips. `ConcurrencyProfile` samples 600 points across a 166.78 s run, a 0.278 s step. The claim that concurrency reached exactly zero rests on an 1.1 s window and is safe at that resolution; the claim that the second dip bottoms at three rather than two is a one-sample distinction and I would not defend it more finely than that.

This is a broker-mediated run, but I cannot verify from the trace what the workers were. The executors are `broker` (the eight host-side rank processes) and `worker` (the 33 claiming processes); this is not a `simulated` or `function` run, and the agent outputs vary per rank in a way the deterministic `synthetic_agent` would not reproduce. What the log cannot show is what ran inside a worker. The latency distribution is mildly suspicious in one respect — all 24 latencies land within 25 ms of a half-second grid — but that is fully explained by the 0.5 s poll interval on the completion path, so I read it as measurement granularity rather than as evidence of a scripted executor.

One run, and the interesting numbers are maxima. The barrier cost, the deepest halo wait, and the load imbalance are all order statistics over eight samples, and order statistics over eight samples are noisy. That rank 6 waited 22.72 s for rank 5 is a fact about this run's particular latency draw; a different draw would move the dip somewhere else in Figure 2 without changing anything structural. The structural claims — that the exchange couples a rank only to its neighbours while the barrier couples it to the population, and that the two therefore scale differently in p — do not depend on which rank drew the long latency, but the specific seconds do.

Context accounting is vacuous here, so half the transport story is untested. Every receive in this harness passes `admit=False` and every rank finalizes with `context_used: 0` and `high_water: 0` against a 120,000-token budget. The run therefore says nothing about admission pressure, eviction, compaction, or whether a context account survives reattachment. Figure 3's empty right-hand panel is not a finding about low context pressure; it is the absence of a measurement.

The trace pre-dates a fix, and I checked only one. The `halo_exchange` instrumentation gap is closed in `ee21d79`, which lands after the commit carrying this trace. Other flagged behaviour in this document may likewise have been fixed since the run, and where I have called something a defect — the missing rank-membership check in `RankRuntime.register` and the double-counted `tokens_deferred` in `RankCost` — I verified it against the source at the current HEAD rather than against the code that produced the trace. Those two are live. Nothing else in the analysis depends on that distinction.